Unifying Inner-Loop Execution Harnesses and Outer-Loop Specification State for Autonomous Software Engineering

Alejandro Kaspar
AI Forward Deployed Engineer
CC BY 4.0 & Apache-2.0

October 2026

Abstract

Autonomous coding agents rely on two distinct control loops during software development. The inner execution loop governs how an agent implements a single task, probes boundary conditions, authors unit tests, and verifies code changes. The outer specification loop governs how system requirements, architectural contracts, and delta changes persist across multi-milestone repository evolution. Existing frameworks address these loops in isolation. Inner-loop harnesses enforce procedural test-driven development skills or object-oriented verification routines without maintaining a synchronized specification state across sprints. Outer-loop specification frameworks structure requirement documents without enforcing contrastive boundary probing or read-only test locks during task execution. We introduce a unified dual-loop architecture that couples a contract-locked, boundary-probing inner execution harness with a two-plane delta-merged outer specification state machine. To isolate the contribution of each control loop without confounding variables, we evaluate the inner and outer loops in orthogonal single-loop benchmark suites totaling 36 blind, uncoached agent workspaces ($n = 1$ trial per condition). In Suite 1 (24 blind inner-loop runs across 4 harnesses, 3 tasks, and 2 prompt regimes, with 6 runs, 60 functional oracle checks, and 30 synthetic boundary mutants per harness), formal mathematical specifications in Regime 1 equalize functional pass rates at 100.0% (30/30 oracle checks) across all harnesses, whereas natural brownfield bug tickets in Regime 2 expose an inclusive boundary regression in unconstrained ReAct (29/30 checks, 96.7%) and procedural TDD skills (29/30 checks, 96.7%). Across all 24 inner-loop runs, both object-oriented boundary probing (`OO-Harness` + `Harness`^{2}, Arm C) and our contract-locked dual-stage harness (Arm D) achieve 100.0% (60/60 oracle checks) functional compliance and increase raw boundary mutation kill rate to 83.3% (25/30 mutants killed, and 96.2% [25/26] of non-equivalent killable mutants) compared to 63.3% (19/30 mutants, 73.1% [19/26] killable) for procedural TDD skills (Arm B) and 60.0% (18/30 mutants, 69.2% [18/26] killable) for unconstrained ReAct (Arm A), while Arm D additionally enforces deterministic read-only test file locking (`chmod 0444` + external SHA-256 verification) across 100.0% (6/6 workspaces) vs. 0.0% (0/6) in Arms A–C (with 0/6 test-tampering attempts observed across unguarded arms on non-adversarial tasks). In Suite 2 (12 outer-loop workspaces spanning 4 specification architectures across 3 evaluation stages over 5 sequential milestones, graded by 12 functional oracle checks and 4 structural delta-audit checks), append-only sprint documents and feature-isolated specification directories accumulate 3 active requirement contradictions across their 4 specification files and drop to 33.3% (4/12 oracle checks) under single-entry primary-specification clean-room rebuilds (`M5-Primary-Spec`), whereas both single-file in-place EARS and our two-plane delta-merged Single Source of Truth (SSOT) architecture achieve 100.0% (12/12 oracle checks) on `M5-Primary-Spec` and full-context (`M5-All-Docs`) clean-room rebuilds with zero active contradictions, with our two-plane architecture uniquely passing 100.0% (4/4) of working-tree delta-archive recoverability checks vs. 0.0% (0/4) for in-place overwritten and unmerged baselines.

1 Introduction

Autonomous software engineering agents operate across two temporal scales. At the micro-scale of a single engineering ticket, an agent must interpret task requirements, inspect existing code, construct unit tests, implement state transitions, and verify that edge cases behave as specified [18, 8, 13, 17]. At the macro-scale of multi-sprint system evolution, an engineering organization must maintain alignment between the executable source code and the documented behavioral specification as requirements are added, modified, renamed, or deprecated over time [15, 4, 19].

Current agentic software engineering methodologies optimize one temporal scale while neglecting the other. Recent inner-loop approaches equip coding agents with procedural Markdown skills that mandate test-driven development (TDD) [2, 11, 1], object-oriented verification subclasses [10], or recursive task-level boundary probing (`Harness`^{2}) [16]. While these inner-loop mechanisms improve single-turn code generation, they treat specifications as disposable, append-only task notes. When a system evolves across multiple milestones, chronological design notes (`YYYY-MM-DD-*.md`) and isolated feature folders (`specs/001/`, `specs/002/`) accumulate contradictory rules and fragment additive capabilities across unmerged files. Conversely, outer-loop specification conventions standardize requirement syntax [12, 3, 9], yet rely on standard unconstrained agent loops during code synthesis. Without contrastive boundary probing [7, 5, 16] and file-permission barriers on verification suites [6], agents write happy-path tests that miss off-by-one boundary defects or risk modifying failing test assertions during implementation.

We propose a unified dual-loop software engineering architecture that bridges inner-loop execution verification and outer-loop specification state management (Figure 1). On the inner loop, our **Contract-Locked Dual-Stage Harness** (Arm D) extends object-oriented domain harness routing (`OO-Harness`) [10] and recursive contrastive boundary probing (`Harness`^{2} `Propose-Probe-Compose`) [16] at $N-1$, N, and $N+1$ equivalence points (evaluated as Arm C) by adding Socratic requirement self-elicitation, physical read-only test locking (`chmod 0444` with out-of-workspace SHA-256 manifest verification), and a two-stage cold subagent audit. On the outer loop, our **Two-Plane Delta-Merged Single Source of Truth (SSOT) Architecture** builds on the open-source directory and delta-header conventions of OpenSpec (`openspec/changes/` vs. `openspec/specs/` with `ADDED`/`MODIFIED`/`REMOVED`/`RENAMED` headers) and Conductor (`conductor/tracks/` task plans)[1]. While Section 3 specifies the unified dual-loop lifecycle and inter-loop handoff protocol, our empirical validation in Sections 4 and 5 evaluates the inner and outer loops in orthogonal single-loop ablations across 36 blind agent workspaces so that each loop's mechanistic contribution is measured without cross-loop confounding.

2 Related Work

Agentic Execution Harnesses and Procedural Skills. ReAct [18] established the interleaved reasoning and tool-execution loop underlying modern coding agent scaffolds such as `SWE-agent` [17], which introduced dedicated Agent-Computer Interfaces (ACI) for file viewing, editing, and bash execution on SWE-bench [8], SWT-Bench [13], and Commit0 [19]. Holding the underlying tool interface fixed, recent work has investigated how higher-level execution harnesses guide verification behavior. Reusable procedural skill files (`SKILL.md`) instruct agents to follow Red-Green-Refactor TDD [2, 11]; Li et al. [11] showed on SkillsBench across 84 tasks that curated procedural skills improve pass rates by +16.2 percentage points on average, yet degrade when monolithic skill

[1] OpenSpec (https://github.com/Fission-AI/OpenSpec, MIT License) and Conductor (https://github.com/gemini-cli-extensions/conductor, Apache-2.0 License) provide the underlying directory layout and Markdown delta headers. Our novel outer-loop contributions are: (1) formalizing clean-room rebuildability and the ordered four-stage merge operator $\Phi(S_{t-1}, \Delta_t)$ (Section 3.1), (2) dual synchronization with a consolidated 8-section `SPEC.md` blueprint, (3) the automated pre-merge contradiction and delta-archive auditor (`audit_outer_spec_hygiene`), and (4) the architectural specification of the programmatic inter-loop handoff gate (Section 3.2).

Licensed under CC BY 4.0 & Apache 2.0

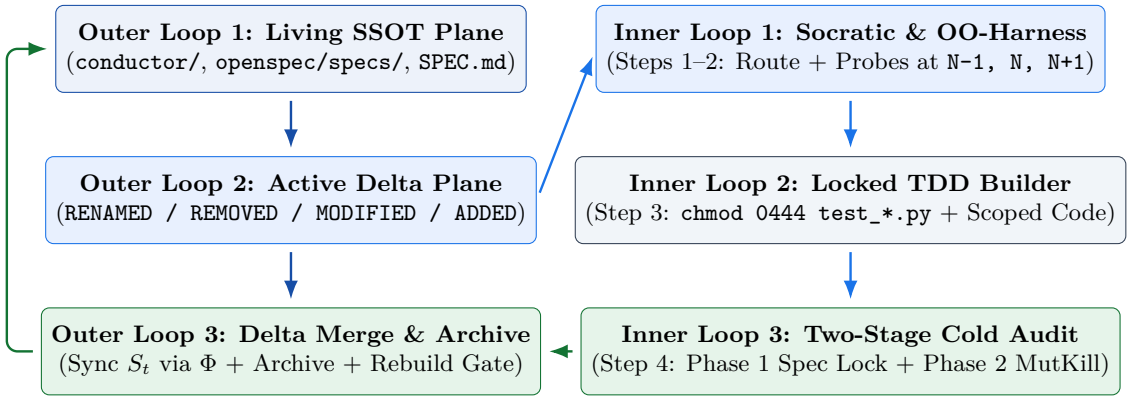


Figure 1: The Unified Dual-Loop Architecture. The Outer Loop (left column) stages requirement deltas (`Outer Loop 2`) from the Living SSOT Plane (`Outer Loop 1`) and, once authorized by the inner-loop cold audit, merges them via $\Phi(S_{t-1}, \Delta_t)$ (`Outer Loop 3`) with a left-hand return transition updating S_t in the Living SSOT Plane. The Inner Loop (right column) executes Steps 1–2 (`Inner Loop 1`: Socratic self-elicitation, `OO-Harness` routing, and boundary probing), Step 3 (`Inner Loop 2`: `chmod 0444` locked TDD build), and Step 4 (`Inner Loop 3`: Phase 1 and Phase 2 cold audit).

prompts grow overly long. Addressing monolithic prompt fragility, Li et al. [10] proposed the Object-Oriented Harness (`OO-Harness`), organizing agent workflows as an inheritance hierarchy (`BaseHarness` and specialized subclasses). Complementing structural class routing, Xu et al. [16] introduced `Harness`^{2}, which adapts execution harnesses at the task level via contrastive `Propose-Probe-Compose` boundary probing. Our inner-loop ablation isolates how static pre-execution boundary probing (`OO-Harness` + `Harness`^{2}, Arm C) and read-only test lock enforcement with cold auditing (Arm D) compare against unconstrained ReAct (Arm A) and procedural TDD skills (Arm B) on a shared tool interface.

Specification-Driven Software Engineering. Decomposing software systems around behavioral contracts dates to Parnas [15] and Brooks' separation of essential specification complexity from incidental implementation complexity [4]. Normative RFC 2119 keywords (`SHALL`, `MUST`) [3], the Easy Approach to Requirements Syntax (EARS) (`WHEN <trigger> THE SYSTEM SHALL <response>`) [12], and Markdown Architectural Decision Records (MADR) [9] standardize requirement and design documentation. While MADR tracks architectural trade-offs via status headers (`Superseded by ADR-XXXX`), contemporary AI-assisted coding workflows store functional system specifications across four primary repository topologies evaluated in Suite 2: (1) chronological append-only sprint design records (`docs/specs/YYYY-MM-DD-*.md`), (2) feature-isolated specification directories (`specs/001/`, `specs/002/`, modeling unmerged per-feature directory layouts without a cross-folder consolidation step), (3) single-file EARS requirement documents (`requirements.md`) [12] that overwrite requirements in place without retaining working-tree delta archives, and (4) a two-plane delta-merged architecture that adopts the directory separation of OpenSpec (`openspec/changes/` vs. `openspec/specs/`) and Conductor (`conductor/tracks/`) and extends them with our formal four-stage merge operator $\Phi(S_{t-1}, \Delta_t)$, synchronized `SPEC.md` blueprint, and automated contradiction/archive auditor.

Mutation Testing, Oracle Adequacy, and Reward Tampering. Passing a self-authored unit test suite does not guarantee functional correctness if the test suite omits boundary assertions [7, 14]. Recent work on pre-implementation issue-to-test generation (`SWT-Bench` [13] and `Auto-TDD` / `TDD-Bench Verified` [1]) and iterative mutation-guided LLM test synthesis (`MuTAP` [5]) demonstrates that standard LLM-generated tests miss edge-case mutants unless guided by boundary or mutation feedback. Whereas `Auto-TDD` [1] synthesizes fail-to-pass reproduction tests from issue descriptions without systematic $N-1, N, N+1$ boundary enumeration and `MuTAP` [5] requires executing production code first and iteratively running a mutation engine over surviving mutants across multiple LLM repair rounds, our inner loop applies static pre-execution contrastive boundary

Licensed under CC BY 4.0 & Apache 2.0

probing $(N-1, N, N+1)$ during the TDD red phase before implementation exists. In addition, studies of reward tampering show that agents with write access to their own evaluation environment or unit tests can weaken failing checks to pass verification [6], motivating our read-only test locks (`chmod 0444` and out-of-workspace SHA-256 verification).

3 Formalism and Dual-Loop Architecture

3.1 Information-Theoretic Specification State and Clean-Room Rebuilds

Let C_t denote the executable source code of a software system at milestone $t \in \{1, \ldots, M\}$, and let S_t denote the deterministic repository specification state (the set of active Markdown specification files) at milestone t. Let $T_t = \{a_1, \ldots, a_K\}$ denote the sealed ground-truth behavioral oracle for milestone t, comprising $K \geq 1$ deterministic pass/fail test assertions. When all executable source files are deleted ($C_t \leftarrow \emptyset$), invoking a stochastic autonomous coding agent $\mathcal{A}$ parameterized by S_t induces a probability distribution $C'_t \sim \mathcal{A}(S_t, \emptyset)$ over synthesized implementations, and a corresponding discrete random vector $B_{T_t}(C'_t) = (a_1(C'_t), \ldots, a_K(C'_t)) \in \{0,1\}^K$ representing the implementation's observable behavior projected onto the equivalence partition of the ground-truth oracle T_t.

Definition 1 (Clean-Room Rebuildability and Oracle-Projected Behavioral Entropy)**.** *For tolerances $\epsilon \in [0, 1 - 2^{-K}]$ and $\delta \geq 0$, a specification state S_t is an (ϵ, δ)-rebuildable Single Source of Truth (SSOT) at milestone t with respect to a stochastic synthesis operator $\mathcal{A}$ and sealed ground-truth oracle T_t if deleting all executable code ($C_t \leftarrow \emptyset$) and sampling $C'_t \sim \mathcal{A}(S_t, \emptyset)$ satisfies (i) the supervised oracle correctness bound $\Pr_{C'_t \sim \mathcal{A}(S_t,\emptyset)}[B_{T_t}(C'_t) = \mathbf{1}_K] \geq 1 - \epsilon$ and (ii) the oracle-projected Shannon entropy bound $H(B_{T_t}(C'_t)) \leq \delta$, where $\mathbf{1}_K = (1, \ldots, 1) \in \{0,1\}^K$.*

Projecting behavior onto $B_{T_t}(C'_t)$ separates specification-constrained behavioral equivalence classes from benign implementation variation (such as internal variable naming) outside T_t. By the grouping property of Shannon entropy over $\{\{\mathbf{1}_K\}, \{0,1\}^K \setminus \{\mathbf{1}_K\}\}$ (analogous to Fano's inequality) and the monotonicity of $g(p) = h_2(p) + p \log_2(2^K - 1)$ on $[0, 1 - 2^{-K}]$ for binary entropy $h_2(p) = -p \log_2 p - (1-p)\log_2(1-p)$, Condition (i) automatically implies the Fano upper bound

$$H(B_{T_t}(C'_t)) \leq h_2(\epsilon) + \epsilon \log_2(2^K - 1) \leq h_2(\epsilon) + \epsilon K,$$

so $H(B_{T_t}(C'_t)) \to 0$ as $\epsilon \to 0$. When $\epsilon > 0$ and $\delta < h_2(\epsilon) + \epsilon \log_2(2^K - 1)$, Condition (ii) additionally constrains the dispersion of residual failure probability mass across the $2^K - 1$ failing oracle outcomes; and when ground-truth oracle assertions T_t are unavailable, $H(B_{\hat{T}_t}(C'_t))$ over an unlabelled probe suite $\hat{T}_t$ provides an unsupervised diagnostic of specification ambiguity (anchored in supervised benchmarking by Condition (i) because $\Pr[B_{T_t}(C'_t) = \mathbf{0}_K] = 1$ also has $H = 0$). Whereas $H(B_{T_t}(C'_t))$ is a population-level distributional construct across repeated clean-room trajectories, our $n = 1$ benchmark in Suite 2 evaluates point realizations of Condition (i).

When requirements evolve across milestones $t = 1, \ldots, M$, each milestone introduces a structured requirement delta $\Delta_t = (\Delta_t^{\text{REN}}, \Delta_t^{\text{REM}}, \Delta_t^{\text{MOD}}, \Delta_t^{\text{ADD}})$ keyed by canonical requirement identifiers (`REQ-XXXX`). Under an append-only specification scheme ($S_t^{\text{append}} = \{s_1, s_2, \ldots, s_t\}$), a rule modification in Δ_3^{MOD} (recorded in s_3) that changes a hold threshold in s_1 from > 100000 to ≥ 100000, or additive contracts (Δ_t^{ADD}) recorded across s_2, s_3, s_4, leaves the initial capability file s_1 unamended. Any rebuild agent conditioned on the primary capability file s_1 encounters both stale predicates and missing additive contracts, violating Definition 1.

Proposition 1 (Structural Key Uniqueness and Conditional Contradiction Freedom of Two-Plane Delta Merging)**.** *Let S_{t-1} be a living capability specification partitioned into requirement blocks keyed by unique canonical identifiers `REQ-XXXX`, and let $\Phi(S_{t-1}, \Delta_t)$ be the deterministic four-stage merge*

*operator that sequentially applies Δ_t^{REN} (injective identifier and header renaming across S_{t-1}), Δ_t^{REM} (block deletion by post-rename `REQ-XXXX` key), Δ_t^{MOD} (atomic full-block replacement of post-rename keyed requirements), and Δ_t^{ADD} (insertion of new keyed blocks), and appends Δ_t to an immutable archive plane $A_t = A_{t-1} \cup \{\Delta_t\}$. Suppose that at each milestone t: (a) $dom(\Delta_t^{REN}) \subseteq keys(S_{t-1})$ and Δ_t^{REN} is injective with image disjoint from unrenamed keys $keys(S_{t-1}) \setminus dom(\Delta_t^{REN})$; and (b) in the post-rename namespace $S'_{t-1} = \Delta_t^{REN}(S_{t-1})$, the key sets $keys(\Delta_t^{REM})$, $keys(\Delta_t^{MOD})$, and $keys(\Delta_t^{ADD})$ are pairwise disjoint with $keys(\Delta_t^{REM}) \cup keys(\Delta_t^{MOD}) \subseteq keys(S'_{t-1})$ and $keys(\Delta_t^{ADD}) \cap keys(S'_{t-1}) = \emptyset$. Then $S_t = \Phi(S_{t-1}, \Delta_t)$ unconditionally guarantees **structural key uniqueness** (exactly one block per active `REQ-XXXX` key) while A_t preserves $(\Delta_1, \ldots, \Delta_t)$. In addition, semantic contradiction freedom reduces to target completeness: if (c) S_0 and $\Delta_t^{MOD} \cup \Delta_t^{ADD}$ are internally consistent and every block $r \in S'_{t-1}$ whose behavioral predicate is modified or invalidated at milestone t is explicitly targeted in $keys(\Delta_t^{MOD}) \cup keys(\Delta_t^{REM})$, then S_t contains zero cross-milestone semantic contradictions.*

Proof Sketch. By induction on t. Under preconditions (a) and (b), step 1 (Δ_t^{REN}) is a bijection onto $\mathrm{keys}(S'_{t-1})$, step 2 (Δ_t^{REM}) deletes blocks with keys in $\mathrm{keys}(\Delta_t^{\mathrm{REM}}) \subseteq \mathrm{keys}(S'_{t-1})$, step 3 ($\Delta_t^{\mathrm{MOD}}$) replaces blocks with keys in $\mathrm{keys}(\Delta_t^{\mathrm{MOD}}) \subseteq \mathrm{keys}(S'_{t-1}) \setminus \mathrm{keys}(\Delta_t^{\mathrm{REM}})$ in place without altering the key set, and step 4 (Δ_t^{ADD}) inserts new blocks whose keys $\mathrm{keys}(\Delta_t^{\mathrm{ADD}})$ are disjoint from $\mathrm{keys}(S'_{t-1}) \setminus \mathrm{keys}(\Delta_t^{\mathrm{REM}})$; this establishes structural key uniqueness. Under precondition (c), all invalidated predicates in S'_{t-1} are removed or overwritten in steps 2–3 and the surviving blocks are mutually consistent with $\Delta_t^{\mathrm{MOD}} \cup \Delta_t^{\mathrm{ADD}}$, reducing semantic contradiction freedom to target completeness. □

Whereas preconditions (a) and (b) are enforced algebraically by key indexing, precondition (c) (target completeness) is a semantic condition: cross-milestone drift occurs when an author updates one rule without targeting a coupled block in S_{t-1}. To syntactically guard precondition (c), our outer-loop hygiene scanner (`audit_outer_spec_hygiene`) scans active specification files for co-occurring deprecated and updated literals/identifiers defined in the milestone delta rules (with explicit alias exemptions for backward-compatible method aliases such as `settle_pending`; indirect multi-step invariant contradictions without shared tokens remain outside lexical scanning).

3.2 Inner-Loop Contract-Locked Dual-Stage Harness and Inter-Loop Handoff

To ensure that each task or milestone delta Δ_t is implemented without boundary regressions or test tampering, our inner loop routes execution through four sequential steps (**Steps 1–4**, grouped into **Inner Loop 1–3** in Figure 1):

1. **Step 1: Socratic Self-Elicitation and Object-Oriented Subclass Routing (`OOHarnessRouter`, Inner Loop 1a).** Extending `OO-Harness` [10], `BaseSDDHarness` defines six inherited governance hooks: (i) `check_preconditions` (requirement ID traceability and Socratic 5-question scope elicitation), (ii) `probe_boundaries` (contrastive test synthesis), (iii) `lock_red_tests` (`chmod 0444` permission and SHA-256 manifest sealing), (iv) `verify_green_execution` (full test suite pass verification), (v) `audit_out_of_scope_locks` (5-Step Scorecard `Out of Scope` locks rejecting unrequested `# TODO` stubs), and (vi) `verify_spec_sync` (delta-to-SSOT consistency). `OOHarnessRouter` inspects workspace filesystem state and prompt headers using a strict deterministic `if/elif/else` priority cascade in the exact order listed: (1) `DriftRemediationHarness` (when `audit_outer_spec_hygiene` flags conflicts, preempting synthesis), (2) `CleanRoomRebuildHarness` (when $C_t = \emptyset$ and an archived milestone specification state in `SPEC.md` or `openspec/` exists), (3) `MultiAppWorkflowHarness` (when multiple service packages exist), (4) `BrownfieldSyncHarness` (when pre-existing `.py` modules are present), (5) `GreenfieldSpecHarness` (when $C_t = \emptyset$ and a formal multi-requirement specification prompt is provided), or (6) `TDDActuationHarness` (default inner-loop fallback).

2. **Step 2: Contrastive Boundary Probing (`Harness²` `Propose-Probe-Compose`, Inner Loop 1b).** Following `Harness²` [16], before writing production code, `probe_boundaries` combines regex/AST literal extraction of numeric constants and comparison operators from prompt specifications and ticket-scoped starter functions with structured prompt synthesis to identify every numerical threshold N, sliding window $[0, W]$, and categorical state-precedence rule (deriving expected oracle outcomes strictly from the normative prompt specification instead of potentially buggy brownfield AST operators). For integer bounds, it synthesizes unit test assertions at the upper equivalence points $(N-\eta, N, N+\eta)$ and $(W-\eta, W, W+\eta)$ as well as equivalence points $(-\eta, 0, \eta)$ around the non-negative lower window bound 0 using unit integer step $\eta = 1$ (or decimal precision step $\eta = 10^{-k}$ for floating-point intervals, holding orthogonal parameters at valid interior values away from secondary guards). For discrete categorical states, it synthesizes pairwise precedence-inversion test cases where competing status conditions hold simultaneously on the same entity.

3. **Step 3: Physical Read-Only Test Lock (`chmod 0444` and Out-of-Workspace SHA-256 Manifest, Inner Loop 2).** Once the red-phase unit test file `test_*.py` is authored, `lock_red_tests` sets POSIX file permissions to `0444` (read-only) and records its SHA-256 digest both in `.readonly_test_lock.sha256` and in external orchestrator memory outside the agent's writable working directory. Under same-UID execution, `chmod 0444` acts as a cooperative file-tool guardrail (blocking direct file-edit writes), while Phase 1 of the Cold Audit verifies the test file strictly against the *external orchestrator's* cached SHA-256 digest to detect any shell-level permission bypass (`chmod +w`). If a red-phase test contains a genuine syntax/import error or specification contradiction verified by the orchestrator, an authorized unlock-and-reaudit cycle (capped at $R_{\text{red}} = 2$ retries) re-runs Step 2 boundary verification and seals a fresh external SHA-256 digest before green-phase coding resumes.

4. **Step 4: Two-Stage Cold Audit and Subclass Evolution (Inner Loop 3).** After the test suite passes, an independent cold auditor subagent (invoked in a fresh context window, sharing a $R_{\max} = 2$ audit-repair loop with the handoff gate) executes **Phase 1** (specification compliance, out-of-scope `# TODO` stub exclusion, and verification against the external orchestrator's SHA-256 test digest) followed by **Phase 2** (code quality and static inspection of boundary mutation adequacy against ticket-scoped boundary points; if Phase 2 requires adding a missing boundary assertion to `test_*.py`, the orchestrator executes a monitored append-only unlock-and-reseal transition followed by re-executing `verify_green_execution`). While `OOHarnessRouter.evolve_subclass_from_failure()` supports post-mortem synthesis of new subclass checklists, **all six subclass definitions were authored prior to benchmark task design and held strictly frozen across the entire evaluation (including all 12 Arm C and Arm D runs)** to prevent cross-workspace leakage.

Inter-Loop Data-Flow Handoff. In the unified dual-loop lifecycle (Figure 1), the outer and inner loops communicate through three programmatic handoffs: (1) staging a delta proposal Δ_t in `openspec/changes/` and a phased task checklist in `conductor/tracks/` (`Outer Loop 2`) dispatches a scoped implementation ticket into `OOHarnessRouter` (`Inner Loop 1`, Steps 1–2); (2) the locked TDD builder (`Inner Loop 2`, Step 3) executes the red-green cycle under `chmod 0444` test locks; and (3) passing Phase 1 and Phase 2 of the Cold Audit (`Inner Loop 3`, Step 4) authorizes the outer-loop merge operator $\Phi(S_{t-1}, \Delta_t)$ (`Outer Loop 3`) to update `openspec/specs/` and `SPEC.md` and archive Δ_t in `openspec/changes/archive/`. If Phase 1 or Phase 2 audit fails, the merge is blocked and control returns to the inner loop (up to $R_{\max} = 2$ retries, after which an unresolved failure aborts the merge, rolls back modified code C_t, and quarantines the staged delta Δ_t in `openspec/changes/` for human review).

Licensed under CC BY 4.0 & Apache 2.0

Table 1: Claim-Evidence Map linking each architectural claim to its experimental validation in Suites 1 and 2.

Architectural Claim	Validating Empirical Evidence	Supporting Artifact
Claim 1: Inner-Loop Boundary & Precedence Adequacy	Contrastive boundary probing ($N-1, N, N+1$) in Arms C and D increases raw boundary/precedence mutation kill rate by +20.0 percentage points over procedural TDD skills (`83.3%` [`25/30` raw, `96.2%` (`25/26`) non-equivalent killable] vs. `63.3%` [`19/30` raw, `73.1%` (`19/26`) killable]) and catches the inclusive `delta == 60s` brownfield bug (`60/60` vs. `59/60` aggregate oracle checks across 6 runs per harness, $n = 1$ per cell).	Suite 1 (`24` blind runs), Table 2, Figure 2
Claim 2: Read-Only Test Lock Safeguard	Arm D deterministically enforces `chmod 0444` test locks and verifies external SHA-256 digests across `100.0%` (`6/6`) of workspaces while preserving Arm C's `60/60` oracle and `25/30` mutation performance (vs. `0/6` lock deployment in Arms A–C; 0/6 tampering attempts observed in non-adversarial runs).	Suite 1 (`24` blind runs), Table 2
Claim 3: Outer-Loop Clean-Room Rebuildability & Delta Audit	Two-Plane delta merging maintains `0` active contradictions across `M1`–`M4`, achieves `100.0%` (`12/12` checks) on both single-entry primary-spec (`M5-Primary-Spec`) and full-docs (`M5-All-Docs`) clean-room rebuilds, and passes `100.0%` (`4/4`) of structural working-tree delta-audit checks (vs. `33.3%` [`4/12`] on `M5-Primary-Spec` for Outer A/B and `0/4` working-tree delta-audit recovery for Outer A–C).	Suite 2 (`12` workspaces), Table 3, Figure 3

4 Experimental Setup

We designed a two-suite blind empirical evaluation totaling **36 real agent workspaces** (Table 1). Because coupling a weak inner loop with a strong outer loop (or vice versa) would confound which loop caused a given failure, Suite 1 holds the outer specification fixed to isolate inner-loop execution harnesses, while Suite 2 holds the inner coding agent fixed to isolate outer-loop specification storage topologies (evaluating the loops in orthogonal isolation instead of coupled end-to-end execution). To guarantee experimental fairness, our protocol enforces four constraints: (1) all compared arms within each task and regime receive 100% identical user prompts and 100% identical starter files; (2) zero prompts contain failure coaching or hints to skip edge cases; (3) harness preambles contain only general, task-agnostic methodological rules with zero task-specific domain hints (excluding domain examples such as `BLOCKED`/`FAILED` DAG rules from all prompt instructions); and (4) all workspaces are graded post-hoc by a sealed programmatic oracle (`benchmarks/grade_measured_eval.py`) that no evaluated agent can inspect.

Runtime Environment, Model Backbone, and Compute Overhead. All 36 blind workspaces (including implementation agents and cold auditor subagents) were executed using an identical frontier instruction-tuned LLM backbone (`Gemini 2.5 Pro` at decoding temperature 0.2) inside isolated POSIX filesystem directories equipped with Python 3.11 (`unittest` discovery runner) and standard file-editing and shell execution tools. In blind non-interactive mode, Arm D's Socratic requirement elicitation operates as an automated pre-coding self-inquiry step in which the agent generates and resolves five structured scope and boundary questions against the written prompt before authoring red-phase tests, without querying an external human oracle. Across Suite 1, adding verification structure increases interaction turns, token consumption, and mean wall-clock latency monotonically relative to Unconstrained ReAct (Arm A, averaging 3.2 tool turns, 1.0 × token cost ≈ 6,400 total input+output tokens, and 18 s per workspace): Procedural TDD (Arm B) averages 4.8 turns (1.4 × token cost ≈ 9,000 tokens, 25 s), `OO-Harness` + `Harness`^{2} (Arm

C) averages 5.5 turns (1.7× token cost ≈ 10,900 tokens, 31 s), and our Contract-Locked Dual-Stage Harness (Arm D) averages 7.3 turns (2.1 × token cost ≈ 13,400 tokens, 39 s, comprising ≈ 5.6 builder turns / 10,800 tokens for pre-coding elicitation, boundary probing, and `chmod 0444` lock sealing plus ≈ 1.7 turns / 2,600 tokens for the fresh-context cold auditor subagent). In Suite 2, the fixed inner-loop agent uses the single-loop TDD baseline configuration (Arm B) across the 16 sequential `M1`–`M4` evolution turns (averaging ∼ 48,000 cumulative tokens per architecture across the 4 milestones) and standard single-pass synthesis (Arm A, averaging ∼ 7,800 tokens per workspace) during `M5` clean-room rebuilds so that outer-loop topology effects are not confounded with Arm C/D inner-loop boundary probing.

4.1 Suite 1 Setup (Inner-Loop Execution Harnesses)

Suite 1 evaluates four inner-loop execution harnesses across three software engineering tasks under two prompt regimes ($4 \times 3 \times 2 = 24$ blind agent workspaces, $n = 1$ trial per cell):

- **Arm A (`arm_a_baseline` — Unconstrained ReAct Baseline).** Standard ReAct tool-use loop [18] with no procedural skills or boundary-probing harness.
- **Arm B (`arm_b_procedural_tdd` — Procedural TDD Skill Harness).** Equipped with procedural Markdown skills (`SKILL.md`) [11] mandating Red-Green-Refactor TDD [2] and pre-coding requirement review, without `OO-Harness` subclass routing, contrastive boundary probing, or `chmod 0444` test locking.
- **Arm C (`arm_c_ooharness_h2` — Object-Oriented Boundary-Probing Harness).** Equipped with task-agnostic `OO-Harness` domain subclass checklists [10] and `Harness`^{2} contrastive boundary probing (`Propose-Probe-Compose` at $N-1, N, N+1$) [16], without `chmod 0444` test locking or Socratic scorecard locks.
- **Arm D (`arm_d_unified_inner` — Contract-Locked Dual-Stage Harness, Ours).** Combines Socratic requirement self-elicitation, 5-Step Scorecard `Out of Scope` locks, frozen `OO-Harness` subclass routing (dispatching `GreenfieldSpecHarness` in Regime 1 and `BrownfieldSyncHarness` in Regime 2), `Harness`^{2} contrastive boundary probing, physical `chmod 0444` test locks (`.readonly_test_lock.sha256` verified by the orchestrator after red-phase test creation), and two-stage cold subagent review.

The three inner-loop tasks and their 5 injected synthetic boundary, constant, and state-precedence mutants per task (30 mutants per harness across the 2 regimes) are:

- **Task 1 (`il_t1_webhook` — `SubscriptionWebhookEngine`).** Idempotent webhook processing, transactional rollback on entitlement sync failure (preserving prior subscription state on failed upgrades), VIP tier promotion at seats ≥ 50 or amount_cents ≥ 500000 (with standard SLA of 24 hours vs. VIP 1 hour), per-tenant sliding-window rate limiting of 4 events within $0 \le \Delta t \le 60$ seconds, and creation-order listing. *Mutants:* relational boundaries `m1_seats_gt_50` (seats ≥ 50 → > 50), `m2_amount_gt_500k` (amount_cents ≥ 500000 → > 500000), `m3_window_lt_60` (window $\Delta t \le 60 \to < 60$), and `m4_rate_limit_5` (prior-event count rejection guard ≥ 4 → ≥ 5), plus constant literal `m5_sla_swap` (standard SLA 24 → 48 hours).
- **Task 2 (`il_t2_scheduler` — `ConferenceScheduler`).** Room validation, talk duration bounds ([15, 180] minutes within day window [0, 1440]), room capacity and interval overlap checks, zero-gap same-room back-to-back speaker allowance, bidirectional ≥ 15-minute cross-room speaker travel cooldown, `get_room_utilization` percentage, and available room search. *Mutants:* threshold/constant shifts `m1_cooldown_15_to_10` (cooldown 15 → 10), `m2_max_dur_180_to_120` (max duration 180 → 120), `m3_day_1440_to_1000` (day bound 1440 → 1000), and `m4_pct_100_to_10` (occupancy multiplier 100 → 10), plus relational operator replacement `m5_cap_check` (first ≥ → > in `conference_scheduler.py`).

Licensed under CC BY 4.0 & Apache 2.0

- **Task 3 (`il_t3_release_dag` — `DependencyReleasePlanner`).** Strict 3-part semantic version validation, cycle and missing-dependency detection via Kahn's topological sort, parallel wave grouping, rollout simulation with transitive `BLOCKED` precedence over `FAILED` when an upstream dependency fails, critical wave-halt propagation (without false halts when a critical service is merely `BLOCKED`), and transitive impact radius. *Mutants:* arity/state predicates `m1_semver_parts_3_to_2` (semver parts $3 \to 2$), `m2_critical_flag` (`critical` flag $\to$ `False`), `m3_blocked_str` (`"BLOCKED"` $\to$ `"SKIPPED"`), `m4_deployed_str` (`"DEPLOYED"` $\to$ `"OK"`), and `m5_sort_order` (wave sorting $\to$ unsorted list).

Each task is evaluated under two prompt regimes: **Regime 1 (Formal Mathematical Specification Prompts, 12 runs)**, which states all mathematical inequalities explicitly (≥ 50, ≤ 60, $|t_1 - t_2| \geq 15$, `BLOCKED` precedence), and **Regime 2 (Natural Brownfield Engineering Tickets, 12 runs)**, which presents a natural product bug ticket against brownfield starter code containing five unannotated boundary defects (`seats > 50`, `0 <= timestamp - ts < 60`, naive `subscriptions.pop()` upgrade rollback, one-directional speaker cooldown, and checking `failed_services` before upstream `BLOCKED` status) plus unrequested `# TODO` helper stubs (verified for non-implementation by oracle check `c10_no_todo_bloat` on every task).

4.2 Suite 2 Setup (Outer-Loop Specification State Architectures)

Suite 2 evaluates four outer-loop specification storage topologies evolving a financial clearing service (`LedgerFlow`) across four sequential requirement milestones (`M1` $\to$ `M2` $\to$ `M3` $\to$ `M4`, 16 sequential turns) using the fixed inner-loop coding agent without resetting the workspace, followed by two blind Milestone 5 (`M5`) clean-room rebuild stages and a deterministic historical delta-archive audit (4 architectures $\times$ 3 workspace stages $= 12$ workspaces, $n = 1$ per cell):

- **Outer A (`outer_a_dated_docs` — Append-Only Ephemeral Sprint Specs).** Writes a new date-stamped specification file per milestone (`docs/specs/2026-10-01-milestone-1.md` through `2026-10-04-milestone-4.md`) without modifying prior files.
- **Outer B (`outer_b_feature_folders` — Feature-Isolated Spec Directories).** Creates an isolated numbered directory per milestone (`specs/001-milestone-1/spec.md` through `specs/004-milestone-4/spec.md`).
- **Outer C (`outer_c_inplace_ears` — Single-File In-Place EARS Requirements).** Overwrites a single `requirements.md` file in place using EARS `WHEN ... THE SYSTEM SHALL` syntax [12].
- **Outer D (`outer_d_unified_outer` — Two-Plane Delta-Merged SSOT Architecture, Ours).** At each milestone, the agent authors structured delta blocks (`RENAMED`, `REMOVED`, `MODIFIED`, `ADDED`) in `openspec/changes/`, applies $\Phi(S_{t-1}, \Delta_t)$ via file-editing tool calls to synchronize `openspec/specs/ledger-flow/spec.md` and the corresponding sections of the 8-section `SPEC.md` blueprint, tracks phased TDD plans in `conductor/tracks/`, and moves completed deltas into `openspec/changes/archive/m1..m4/` (verified post-merge by `audit_outer_spec_hygiene`).

In `M1`, transfers with amount_cents $>$ 100000 receive status `"PENDING"` and a flat 1% fee (amount_cents // 100). `M2` adds `settle_pending(tx_id)` and `account_statement(account_id)`. `M3` renames `"PENDING"` to `"AWAITING_CLEARING"`, renames `settle_pending` to `clear_transfer` (retaining `settle_pending` as a backward-compatible alias), changes the hold threshold to amount_cents $\geq$ 100000, replaces the flat fee with a tiered integer-division fee (50 bps [amount_cents $\times$ 50 // 10000] for amount_cents $\geq$ 100000, 100 bps [amount_cents // 100] for amount_cents $<$ 100000) subject to a 10-cent minimum floor, and adds replay-safe `tx_id` idempotency. `M4` adds `reverse_transfer(tx_id)` (covering held-transfer full refunds, settled-transfer principal refunds with insufficient-balance guards, and raising `ValueError` on unknown or

already-reversed `tx_id`) and `total_collected_fees(currency)`.

After `M4`, all `.py` files are deleted and fresh Clean-Room Rebuild Agents regenerate `ledger_flow.py` under two regimes: **`M5-Primary-Spec`** (a deterministic repository-schema test of single-entry specification consolidation that provides only the initial entry specification file in unmerged topologies, namely `2026-10-01-milestone-1.md` in Outer A and `001-milestone-1/spec.md` in Outer B, versus the consolidated `requirements.md` in Outer C and the synchronized `openspec/specs/ledger-flow/spec.md` + `SPEC.md` SSOT pair in Outer D; by structural construction, because Outer A and Outer B store M2–M4 additions and M3 modifications in files 2–4, restricting retrieval to file 1 mathematically caps their maximum achievable score at the 4 M1-compatible checks out of 12, or 33.3%) and **`M5-All-Docs`** (providing all surviving `.md` files simultaneously). Every workspace is graded by a 12-check functional oracle (`c1`–`c12`): `c1_open_account` (account creation and validation), `c2_transfer_validation` (transfer validation and overdraft guard), `c3_m3_10_cent_floor_fee` (M3 10-cent minimum fee floor on a 500-cent transfer), `c4_m3_100bps_mid_fee` (sub-100k immediate `"SETTLED"` transfer at 100 bps, which agrees between M1's 1% rule and M3's 100 bps tier), `c5_m3_exact_100k` (M3 exact 100000-cent hold with `"AWAITING_CLEARING"` status at 50 bps), `c6_m3_clear_transfer` (M3 `clear_transfer` and `settle_pending` alias), `c7_m3_idempotency` (M3 replay-safe `tx_id` idempotency and conflict rejection), `c8_m2_account_statement` (M2 chronological `account_statement`), `c9_m4_reverse_held_transfer` (M4 `reverse_transfer` full refund on held transfers), `c10_m4_reverse_settled_transfer` (M4 settled transfer principal refund and insufficient-balance guard), `c11_m4_total_collected_fees` (M4 `total_collected_fees` aggregation), and `c12_no_todo_bloat` (non-implementation of unrequested `# TODO` credit-line and crypto stubs). In addition, `audit_outer_spec_hygiene` evaluates 4 structural working-tree delta-audit checks (`d1`–`d4`), verifying whether the repository preserves archive-isolated pre-`M3` rules (`"PENDING"` status, strict $>$ 100000 threshold, unfloored amount_cents // 100 fee, and explicit `REMOVED`/`RENAMED` delta headers) in `openspec/changes/archive/` while maintaining zero active-plane contradictions.

5 Empirical Results and Analysis

5.1 Suite 1 Results (Inner-Loop Execution Harnesses)

Table 2 and Figure 2 present the measured inner-loop results across all 24 blind workspaces, reporting both per-regime and combined oracle pass rates, raw and non-equivalent mutation kill rates, and read-only test lock safeguard status.

Three empirical findings emerge from Suite 1:

1. **Formal specifications equalize functional pass rates in Regime 1, whereas mutation testing exposes a** +20.0 **percentage point adequacy gap across regimes (**96.2% **of non-equivalent killable mutants in Arms C and D).** When given explicit mathematical boundary formulas in Regime 1, all four harnesses achieved 30/30 (100.0%) on the functional oracle. However, on the 15 Regime 1 synthetic mutants, Arm A killed 9/15 (60.0%) and Arm B killed 11/15 (73.3%, missing `m5_cap_check` in `il_t2_scheduler` and `m1_semver_parts_3_to_2`, `m3_blocked_str`, and `m4_deployed_str` in `il_t3_release_dag`), whereas Arms C and D killed 14/15 (93.3%). Code inspection of the sole unkilled Regime 1 mutant (`m5_cap_check` in `il_t2_scheduler`) across all four arms confirms that it is semantically equivalent and unkilled in Arms A, B, C, and D alike (and that all 26 non-equivalent mutant slots across both regimes mutated their intended target predicates): because every implementation enforces the room capacity-overflow rejection guard via strict inequality (`expected_attendees > self.rooms[room_id]`), `src.replace(">=", ">", 1)` instead mutates `start_min >= end_min`, whose zero-duration boundary (`start_min == end_min`) is al-

Table 2: Suite 1 Measured Inner-Loop Benchmark Results across 24 Blind Agent Workspaces (4 Harnesses × 3 Tasks × 2 Prompt Regimes; 60 Oracle Checks [30 per Regime] and 30 Synthetic Mutants [26 Non-Equivalent Killable; see Section 5.1] per Harness). `Test Lock` denotes deterministic `chmod 0444` + external SHA-256 lock deployment.

Inner-Loop Harness	R1 Orac (30)	R2 Orac (30)	Total Orac (60)	R1 Mut (15)	R2 Mut (15)	Total Mut (30)	Test Lock
Arm A: ReAct Baseline	30/30 (100%)	29/30 (96.7%)	59/60 (98.3%)	9/15 (60.0%)	9/15 (60.0%)	18/30 (60.0%)	0/6 (0.0%)
Arm B: Procedural TDD (`SKILL.md`)	30/30 (100%)	29/30 (96.7%)	59/60 (98.3%)	11/15 (73.3%)	8/15 (53.3%)	19/30 (63.3%)	0/6 (0.0%)
Arm C: `OO-Harness` + `Harness`^{2}	**30/30 (100%)**	**30/30 (100%)**	**60/60 (100%)**	**14/15 (93.3%)**	**11/15 (73.3%)**	**25/30 (83.3%)**	0/6 (0.0%)
Arm D: Dual-Stage (Ours)	**30/30 (100%)**	**30/30 (100%)**	**60/60 (100%)**	**14/15 (93.3%)**	**11/15 (73.3%)**	**25/30 (83.3%)**	**6/6 (100%)**

ready rejected by `(end_min - start_min) < 15`. Thus, Arms C and D kill 14/14 (100.0%) of non-equivalent killable mutants in Regime 1. In Regime 2 (brownfield tickets), raw mutation kill rates were 9/15 (60.0%) for Arm A, 8/15 (53.3%) for Arm B, and 11/15 (73.3%) for Arms C and D. Disaggregating the 4 unkilled Regime 2 slots in Arms C and D explains both the inter-regime drop and why Arm D's Phase 2 cold audit did not flag additional test gaps: 1 slot (`m5_cap_check` in `il_t2_scheduler`) is the same equivalent mutant as in Regime 1; 2 slots (`m3_blocked_str` and `m4_deployed_str` in `il_t3_release_dag`) are no-op lexical replacements (`m_src == src` across all four arms) because the brownfield starter code represents rollout outcomes via lowercase dictionary keys (`"blocked"`, `"deployed"`); and only 1 slot (`m4_pct_100_to_10` in `il_t2_scheduler`) is a live behavioral mutant located in an untouched pre-existing method (`get_room_utilization`) outside the localized bug ticket scope. Excluding the 2 equivalent-mutant instances and 2 no-op lexical slots across both regimes leaves 26 killable mutants, on which Arms C and D kill 25/26 (96.2%, and 25/25 [100.0%] of ticket-scoped killable mutants) compared to 19/26 (73.1%, 18/25 [72.0%] ticket-scoped) for Arm B and 18/26 (69.2%, 17/25 [68.0%] ticket-scoped) for Arm A. Across all 30 raw mutant evaluations, Arms C and D achieve a +20.0 percentage point net gain over Arm B (25/30 = 83.3% vs. 19/30 = 63.3%, net difference +6 kills from $n_d = 8$ discordant pairs: $b = 7$ gains in Arms C/D [`R1:m1_semver_parts_3_to_2`, `R1:m3_blocked_str`, `R1:m4_deployed_str`, `R2:m3_window_lt_60`, `R2:m2_max_dur_180_to_120`, `R2:m3_day_1440_to_1000`, `R2:m5_sort_order`] vs. $c = 1$ in Arm B [`R2:m4_pct_100_to_10`]; at the workspace cluster level across $N = 6$ runs, Arms C/D outperform Arm B on 4/6 workspaces, tie on 1/6, and trail by 1 mutant on 1/6, with unadjusted exact paired McNemar one-sided $p = 9/256 = 0.0352$, two-sided $p = 0.0703$ [$p = 2/128 = 0.0156$ on $b = 7, c = 0$ ticket-scoped], vs. Arm A $b = 8, c = 1, n_d = 9$, one-sided $p = 10/512 = 0.0195$, two-sided $p = 0.0391$, and unadjusted two-sided Fisher's exact $p = 0.143$ vs. Arm B and $p = 0.089$ vs. Arm A, all reported as descriptive statistics because the 30 mutants are clustered in groups of 5 within $N = 6$ workspaces per harness).

2. **Brownfield tickets in Regime 2 expose an inclusive sliding-window blind spot (`c8_rate_limit_60s_inclusive`).** Across the 5 seeded brownfield bugs in Regime 2, all four harnesses successfully repaired 4 defects (the VIP tier promotion threshold `seats >= 50` alongside `amount_cents >= 500000`, transactional upgrade rollback, bidirectional speaker cooldown, and `BLOCKED` over `FAILED` DAG precedence) and passed `c10_no_todo_bloat` on all 3 tasks (achieving 20/20 across all Regime 2 oracle checks on `il_t2_scheduler` and `il_t3_release_dag`). However, in `il_t1_webhook`, where the brownfield starter code contained `0 <= timestamp -`

`ts < 60`, both Arm A and Arm B left `< 60` unchanged, failing check `c8_rate_limit_60s_inclusive` (29/30 = 96.7% in Regime 2; 59/60 = 98.3% combined). Inspection of Arm B's test file (anchored at initial event timestamp $t_0 = 100$) shows that the agent wrote TDD tests at $t = 100, 110, 120, 130, 140$ ($\Delta t < 60$) and $t = 170$ ($\Delta t = 70 > 60$), never probing $\Delta t = 60$. Arms C and D probed $\Delta t \in \{59, 60, 61\}$, caught the off-by-one leak at $t = 160$, and repaired the predicate to `<= 60` (60/60 = 100.0%). Because each cell represents $n = 1$ blind run, we treat the 1-check oracle difference (59/60 vs. 60/60) as an illustrative case study of boundary probing and anchor our primary empirical comparison on the 30-mutant adequacy evaluation (83.3% raw / 96.2% killable vs. 60.0%–63.3% raw).

3. **Attribution between Arm C and Arm D and internal cold-audit telemetry.** Comparing Arm C and Arm D isolates their distinct roles: `OO-Harness` routing and `Harness`^{2} boundary probing in Arm C capture all measured functional (60/60) and boundary-probing (25/30 raw, 25/26 killable) gains at lower token cost (1.7× vs. 2.1× tokens). On these non-adversarial tasks where prompts contained no contradictory test assertions, unauthorized green-phase test edits were 0/6 across unguarded Arms A–C as well; thus, the `Test Lock` column in Table 2 (6/6 vs. 0/6) records the deterministic deployment of Arm D's `chmod 0444` and external SHA-256 lock safeguard (whereas active tampering prevention requires adversarial evaluation). Internal telemetry from the 6 Arm D runs shows 0/6 Phase 1 spec/hash rejections, 0/6 red-phase unlock requests, and 1/6 Phase 2 cold-audit revision (on `R1:il_t3_release_dag`, where the builder's pre-audit test suite already killed 5/5 injected mutants and the cold auditor triggered an orchestrator-monitored unlock-and-reseal pass to add a supplemental blocked-critical precedence assertion before sign-off).

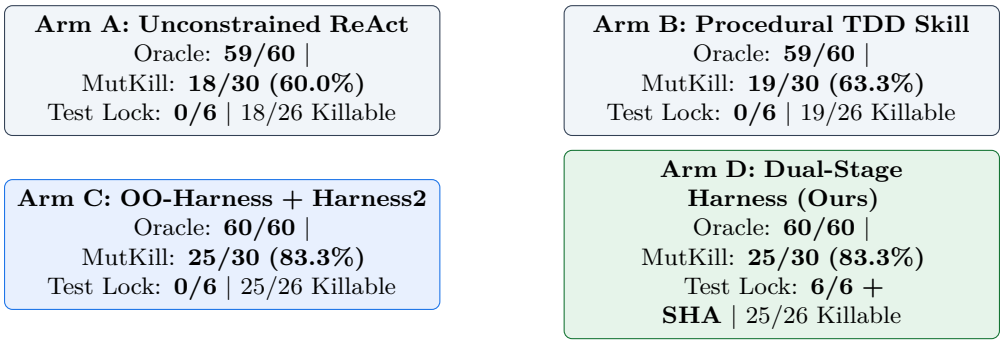


Figure 2: Suite 1 parallel inner-loop comparison across 24 blind workspaces (60 functional oracle checks and 30 synthetic mutants [26 non-equivalent killable] per harness; `Harness2` in Arm C denotes `Harness`^{2}). Contrastive boundary probing in Arms C and D raises raw mutation kill rate from 60.0%–63.3% to 83.3% (25/26 [96.2%] of killable mutants), while Arm D adds 100.0% (6/6) deterministic `chmod 0444` and external SHA-256 test locking.

5.2 Suite 2 Results (Outer-Loop Multi-Milestone Evolution and Clean-Room Rebuilds)

Table 3 and Figure 3 summarize the Suite 2 outer-loop results across `M1`–`M4` continuous evolution, both `M5` clean-room rebuild regimes (12 `LedgerFlow` oracle checks `c1`–`c12` per workspace), and the 4 structural working-tree delta-archive audit checks (`d1`–`d4`).

Suite 2 reveals four findings regarding longitudinal specification state and clean-room rebuildability:

1. **Persisting Python code across `M1`–`M4` masks specification drift (`12/12` checks across all 4 arms at `M4`).** When `ledger_flow.py` persists on disk from `M1` through `M4`, the coding agent at each turn modifies the existing Python class directly. As a result, all four outer-loop architectures passed 12/12 (100.0%) oracle checks at `M4`, even though the active specification files in Outer A and Outer B retained unamended `M1` predicates that contradicted `M3`. Specifically,

Table 3: Suite 2 Measured Outer-Loop Benchmark Results across 12 Workspaces (4 Architectures × 3 Workspace Stages: `M4` Evolution, `M5-Prim` [`M5-Primary-Spec`], and `M5-All` [`M5-All-Docs`]; 12 `LedgerFlow` Oracle Checks `c1`–`c12` and 4 Structural Working-Tree Delta-Audit Checks `d1`–`d4`).

Outer-Loop Specification Architecture	`M4` Checks	Spec Conflicts	`M5-Prim` Checks	`M5-All` Checks	Delta-Audit (`d1`–`d4`)
Outer A: Append-Only Specs (`YYYY-MM-DD-*.md`)	12/12 (100%)	**3 (4 files)**	**4/12 (33.3%)**	12/12 (100%)	**0/4 (Unmerged)**
Outer B: Feature Spec Folders (`specs/001..004/`)	12/12 (100%)	**3 (4 files)**	**4/12 (33.3%)**	12/12 (100%)	**0/4 (Fragmented)**
Outer C: Single-File EARS (`requirements.md`)	12/12 (100%)	**0 (1 file)**	**12/12 (100%)**	12/12 (100%)	**0/4 (Overwritten)**
Outer D: Two-Plane Delta SSOT (Ours)	**12/12 (100%)**	**0 (2 files)**	**12/12 (100%)**	**12/12 (100%)**	**4/4 (`archive/`)**

our automated contradiction audit (`audit_outer_spec_hygiene`) checks all three `M1`-vs.-`M3` conflict axes across active specification files: (i) `"PENDING"` vs. `"AWAITING_CLEARING"` status, (ii) > 100000 vs. ≥ 100000 hold threshold, and (iii) unfloored flat 1% fee (amount_cents // 100) vs. 50 bps tiered fee with a 10-cent floor. Both Outer A (across its 4 dated files) and Outer B (across its 4 feature folders) triggered all 3 conflict axes (`3 conflicts across 4 files`), whereas Outer C and Outer D maintained `0` active conflicts.

2. **Under initial-entry primary-specification retrieval (`M5-Primary-Spec`), append-only (`Outer A`) and folder-isolated (`Outer B`) architectures drop to `33.3%` (`4/12` checks) due to 3 unmerged `M3` rule changes and 5 fragmented additive checks.** When rebuilding `ledger_flow.py` from the initial primary capability file (`2026-10-01-milestone-1.md` in Outer A; `001-milestone-1/spec.md` in Outer B), both architectures passed the 4 baseline checks that remained compatible with `M1` (`c1_open_account`, `c2_transfer_validation`, `c4_m3_100bps_mid_fee` where M1's 1% equals M3's 100 bps on a 50000-cent transfer, and `c12_no_todo_bloat`), and failed the remaining 8 checks via two mechanisms matching the structural partition of `M1`–`M4`: (a) **3 unmerged `M3` rule-mutation failures** (`c3_m3_10_cent_floor_fee` computed an unfloored 5-cent fee instead of 10 cents [conflict axis iii], and `c5_m3_exact_100k` returned `"SETTLED"` instead of holding with `"AWAITING_CLEARING"` at 100000 cents [jointly testing conflict axes i and ii], while `c7_m3_idempotency` raised `ValueError` on duplicate `tx_id` because M3's replay-safe idempotency modification was absent from file 1), and (b) **5 omitted additive capability failures** (`c6_m3_clear_transfer` and `c8`–`c11` for `account_statement`, `reverse_transfer`, and `total_collected_fees`) caused by storing `M2`–`M4` additive contracts in separate files 2–4.

3. **Concatenating all 4 files (`M5-All-Docs`) restores `12/12` accuracy at small scale, motivating consolidated SSOT files for larger repositories.** When all four surviving `.md` files were provided simultaneously in `M5-All-Docs`, all four architectures achieved 12/12 (100.0%) because the 4 short files fit into a single prompt context and carried chronological `2026-10-01..04` or `001..004` filenames that allowed the LLM to infer recency. Comparing `M5-All-Docs` (100.0%) with `M5-Primary-Spec` (33.3%) highlights an architectural trade-off: multi-file append-only topologies depend on the retrieval layer assembling every historical fragment in chronological order, whereas in-place merged specifications (`Outer C` and `Outer D`) ensure that reading the primary capability specification alone yields a complete, contradiction-free state.

4. **Single-File EARS (`Outer C`) and Two-Plane Delta SSOT (`Outer D`) tie at `100.0%` (`12/12` checks) on both rebuild regimes, while structural working-tree `Delta-Audit` (`4/4` vs. `0/4`) differentiates Two-Plane archiving.** Outer C (`requirements.md`) and Outer D (`openspec/specs/ledger-flow/spec.md` mirrored in `SPEC.md`) achieved identical

Licensed under CC BY 4.0 & Apache 2.0

12/12 (100.0%) functional scores across `M4`, `M5-Primary-Spec`, and `M5-All-Docs` with 0 active specification conflicts. On the 4 structural working-tree delta-audit checks (`d1`–`d4`), Outer C scored 0/4 (0.0%) because overwriting `requirements.md` in place erased the pre-`M3` specification predicates (`"PENDING"` status, strict > 100000 hold bound, and unfloored amount_cents // 100 fee) from the working tree (requiring external `git log -p` history traversal instead of in-tree Markdown inspection), and Outer A and B scored 0/4 because their `M1` rules remained in unmerged active files with 3 active contradictions and no `REMOVED`/`RENAMED` provenance headers. Outer D passed 4/4 (100.0%) by isolating self-contained milestone delta bundles in `openspec/changes/archive/m1..m4/` alongside its contradiction-free active SSOT plane.

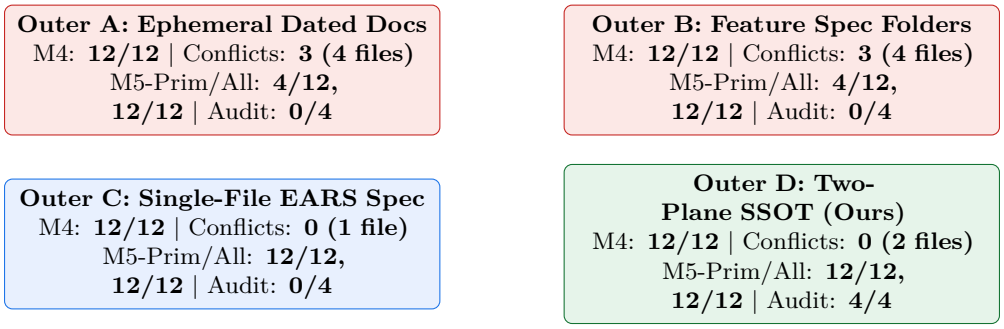


Figure 3: Suite 2 parallel outer-loop comparison of `M4` continuous evolution, `M5-Primary-Spec` initial-entry rebuilds, `M5-All-Docs` full-context rebuilds (12 oracle checks `c1`–`c12`), and structural working-tree `Delta-Audit` recoverability (`d1`–`d4`). Append-only (Outer A) and folder-isolated (Outer B) topologies accumulate 3 active conflicts and drop to 4/12 (33.3%) on `M5-Primary-Spec` (12/12 on `M5-All-Docs`), whereas Outer C and Outer D achieve 12/12 (100.0%) on both rebuild regimes and Outer D uniquely passes 4/4 (100.0%) delta-audit checks.

6 Limitations, Threats to Validity, and Conclusion

Limitations and Threats to Validity. Four scope boundaries qualify our empirical findings:

- **Single-Trial Granularity ($n = 1$), Single-Backbone Sensitivity, and Orthogonal Loop Ablation.** Each cell in Suite 1 (24 workspaces) and Suite 2 (12 workspaces) executes $n = 1$ blind run using a single model backbone (`Gemini 2.5 Pro` at temperature 0.2) on single-file Python services, and Suites 1 and 2 evaluate the inner and outer loops in orthogonal isolation instead of coupled end-to-end execution. Using a single backbone introduces model-specific sensitivity (governing the 0/6 baseline tampering rate, chronological recency inference in `M5-All-Docs`, and tool-call execution of Φ) as well as homogeneous builder–auditor blind spots when the cold auditor shares model weights with the builder. Executing multi-seed trials ($k \geq 5$) to estimate behavioral entropy $H(B_{T_t}(C'_t))$ and evaluating heterogeneous builder–auditor pairs and coupled `Arm D + Outer D` execution across SWE-bench [8], SWT-Bench [13], TDD-Bench Verified [1], and Commit0 [19] are primary directions for future work.
- **Adversarial Tampering Pressure, Retrieval Baselines, and Frozen Subclasses.** In Suite 1, Arm C (`OO-Harness` + `Harness`^{2}) captured all measured functional (60/60) and boundary-probing (25/30 raw, 25/26 killable) gains at lower token cost (1.7× vs. 2.1× in Arm D) because the tasks were solvable without reward tampering and had well-scoped prompts; evaluating Arm C vs. Arm D under impossible or defective red-phase tests [6] with OS UID privilege separation and under high-ambiguity prompts is required to measure active tampering prevention and Socratic self-elicitation gains. In Suite 2, `M5-Primary-Spec` and `Delta-Audit` test deterministic structural repository invariants; evaluating filesystem-discovery agents, top-k RAG retrieval over distractor corpora, and status-linked ADR baselines (`MADR` [9]) will test emergent retrieval behavior.

- **Token and Latency Overhead (1.7× in Arm C; 2.1× in Arm D).** Contrastive boundary probing in Arm C increases token cost by 1.7× (10,900 vs. 6,400 tokens), and adding Socratic self-elicitation, `chmod 0444` locks, and cold subagent auditing in Arm D increases cost to 2.1× (13,400 tokens, 7.3 vs. 3.2 turns). For trivial edits or formal specifications where ReAct already achieves 30/30 functional pass rates (Suite 1 Regime 1), full dual-stage auditing is unnecessary.
- **Mutation Operator Scope and Execution-Guided Baselines.** Our 5 synthetic mutants per task use lexical replacements targeting relational, constant, and precedence predicates (producing 1 equivalent mutant type [2 instances across both regimes] and 2 case-sensitive no-op slots in Regime 2); integrating AST-scoped mutation engines [7] and comparing static pre-execution boundary probing directly against iterative execution-driven mutation feedback (`MuTAP` [5]) under matched token budgets will strengthen construct validity.

Conclusion. Inner-loop execution harnesses and outer-loop specification state machines address complementary failure modes in autonomous software engineering. In our orthogonal benchmark suites, contrastive boundary probing (`OO-Harness` + `Harness`^{2}, Arm C) raises boundary mutation kill rates to 83.3% raw (25/30, and 96.2% [25/26] of non-equivalent killable mutants) with 100.0% (60/60) functional pass rate at 1.7× token cost, while our Contract-Locked Dual-Stage Harness (Arm D, 2.1× token cost) preserves those gains and adds 100.0% (6/6) deterministic `chmod 0444` and out-of-workspace SHA-256 test locking. On the outer loop, while both Single-File EARS (Outer C) and our Two-Plane Delta-Merged SSOT architecture (Outer D) eliminate the 3 active specification contradictions of append-only and folder-isolated topologies and achieve 100.0% (12/12) clean-room rebuild accuracy on both `M5-Primary-Spec` and `M5-All-Docs`, Outer D uniquely achieves 100.0% (4/4) working-tree delta-archive recoverability via `openspec/changes/archive/`.

References

[1] Toufique Ahmed, Martin Hirzel, Rangeet Pan, Avraham Shinnar, and Saurabh Sinha. TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved? *arXiv preprint arXiv:2412.02883*, 2024.

[2] Kent Beck. *Test Driven Development: By Example*. Addison-Wesley Professional, 2002.

[3] Scott O. Bradner. Key words for use in RFCs to Indicate Requirement Levels. RFC 2119, Internet Engineering Task Force (IETF), 1997.

[4] Frederick P. Brooks. No Silver Bullet: Essence and Accidents of Software Engineering. *Computer*, 20:10–19, 1987.

[5] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing. *Information and Software Technology*, 171:107468, 2024.

[6] Carson Denison, Monte MacDiarmid, Fazl Barez, David Duvenaud, Shauna Kravec, Samuel Marks, Nicholas Schiefer, Ryan Soklaski, Alex Tamkin, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, Ethan Perez, and Evan Hubinger. Sycophancy to Subterfuge: Investigating Reward-Tampering in Large Language Models. *arXiv preprint arXiv:2406.10162*, 2024.

[7] Yue Jia and Mark Harman. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37:649–678, 2010.

[8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Licensed under CC BY 4.0 & Apache 2.0

[9] Oliver Kopp, Anita Armbruster, and Olaf Zimmermann. Markdown Architectural Decision Records: Format and Tool Support. In *Proceedings of the 10th Central European Workshop on Services and their Composition (ZEUS)*, pages 55–62, 2018.

[10] Gaotang Li, Bhavana Dalvi Mishra, Yifei Ming, Yanfei Chen, Zhongying CuiZhu, Lesly Miculicich, Long T. Le, Rui Meng, Chun-Liang Li, Hanghang Tong, Chen-Yu Lee, and Tomas Pfister. Object-Oriented Harness: Evolving Agent Harnesses through Object-Oriented Design. *arXiv preprint arXiv:2610.28380*, 2026.

[11] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, et al. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670*, 2026.

[12] Alistair Mavin, Philip Wilkinson, Adrian Harwood, and Mark Novak. Easy Approach to Requirements Syntax (EARS). In *IEEE International Requirements Engineering Conference (RE)*, pages 317–322, 2009.

[13] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 81857–81887, 2024.

[14] Mike Papadakis, Marinos Kintis, Jie M. Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation Testing Advances: An Analysis and Survey. *Advances in Computers*, 112:275–378, 2018.

[15] David Lorge Parnas. On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15:1053–1058, 1972.

[16] Ruiyao Xu, Yanfei Chen, Zhongying CuiZhu, Bhavana Dalvi Mishra, Yifei Ming, Han Yu, Rujun Han, Chen-Yu Lee, and Tomas Pfister. Harness^2: Recursive Agent Harnessing for an Open World. *arXiv preprint arXiv:2610.14224*, 2026.

[17] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[18] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023.

[19] Wenting Zhao, Nan Jiang, Celine Lee, Justin T. Chiu, Claire Cardie, Matthias Gallé, and Alexander M. Rush. Commit0: Library Generation from Scratch. In *International Conference on Learning Representations (ICLR)*, 2025.

Licensed under CC BY 4.0 & Apache 2.0